CS 223 Digital Design

Lab 2: Decoders and Multiplexers

Name: Derya POLAT

ID: 22403046

Date: 2.11.2025

1. **Multiplexer**

3-to-1 Multiplexer

Data Inputs: D_0, D_1, D_2

Selects: s_0, s_1

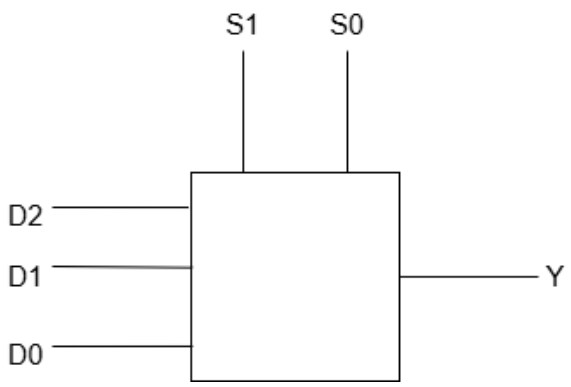
Boolean Equation with AND:

$$Y = \overline{s_0}.\overline{s_1}.D_0 + s_0.\overline{s_1}.D_1 + s_1.\overline{s_0}.D_2$$

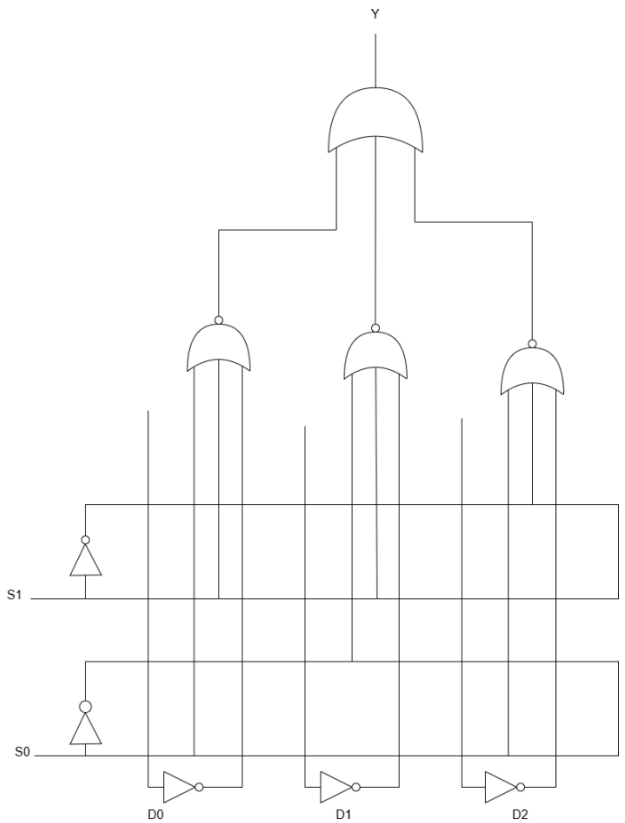
Boolean Equation Using OR and NOT:

$$Y = \overline{(s_0 + s_1 + \overline{D_0})} + \overline{(\overline{s_0} + s_1 + \overline{D_1})} + \overline{(\overline{s_1} + s_0 + \overline{D_2})}$$

Graph of a 3-to-1 Multiplexer:



Logic Diagram:



Truth Table:

s_1	s_0	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	

3-to-1 Multiplexer's Behavioral SystemVerilog

```
module mux31 (
   input logic D0, D1, D2,
   input logic s1, s0,
   output logic Y
);
   always_comb begin
      unique case ({s1,s0})
         2'b00: Y = D0;
         2'b01: Y = D1;
         2'b10: Y = D2;
         default: Y = 1'b0;
      endcase
   end
endmodule
```

3-to-1 Multiplexer's TestBench

```
module tb_mux31;
   logic D0,D1,D2,s1,s0,Y;
   mux31 dut(.D0, .D1, .D2, .s1, .s0, .Y);
   initial begin
      foreach ( {s1,s0} ) ; // no-op, for lint happiness
      for (int sel=0; sel<4; sel++) begin
         {s1,s0} = sel[1:0];
         for (int v=0; v<8; v++) begin
            {D2,D1,D0} = v[2:0];
            #1;
         end
      end
      $finish;
   end
endmodule
```

4-to-1 Multiplexer

Data Inputs: D_0 (00), D_1(01), D_2(10), D_3(11)

Selects: s_0, s_1

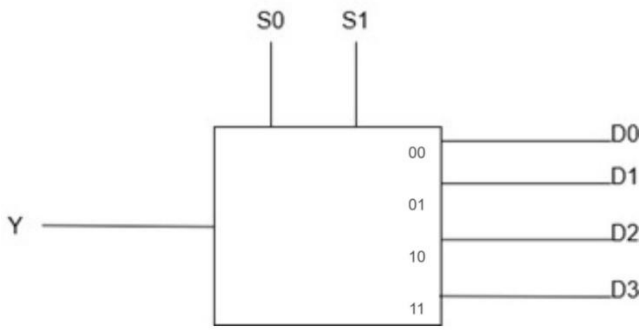
Boolean Equation:

$$Y = \overline{s_0}.\overline{s_1}.D_0 + s_0.\overline{s_1}.D_1 + \overline{s_0}.s_1.D_2 + s_0.s_1.D_3$$

Truth Table:

s_1	s_0	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

Graph of a 4-to-1 Multiplexer:



4-to-1 Multiplexer Built from Three 2-to-1 Multiplexers' Behavioral SystemVerilog

```
module mux21 (
input logic A, B, sel,

            output logic Y
);
   assign Y = sel ? B : A;
endmodule
module mux4_ 2 (
   input  logic D0, D1, D2, D3,
   input  logic s1, s0,
   output logic Y
```

```
);
   logic y01, y23;
   mux21 m01(.A(D0), .B(D1), .sel(s0), .Y(y01));
   mux21 m23(.A(D2), .B(D3), .sel(s0), .Y(y23));
   mux21 mout(.A(y01), .B(y23), .sel(s1), .Y(Y));
endmodule
```

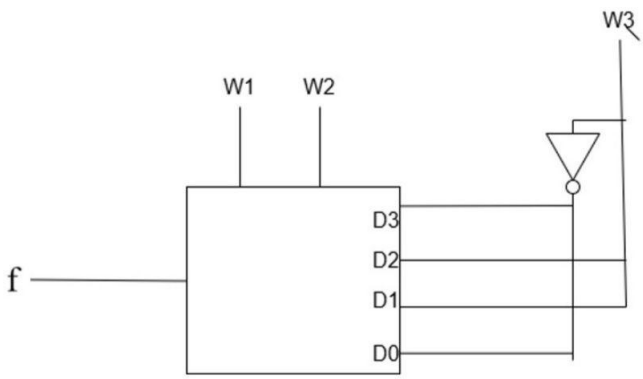
4-to-1 Multiplexer Built from Three 2-to-1 Multiplexers' TestBench

```
module tb_mux4_2;
   logic D0,D1,D2,D3,s1,s0,Y;
   mux4_2 dut(.D0,.D1,.D2,.D3,.s1,.s0,.Y);
   initial begin
      for (int sel=0; sel<4; sel++) begin
         {s1,s0} = sel[1:0];
         for (int v=0; v<16; v++) begin
            {D3,D2,D1,D0} = v[3:0];
            #1;
         end
      end
      $finish;
   end
endmodule
```

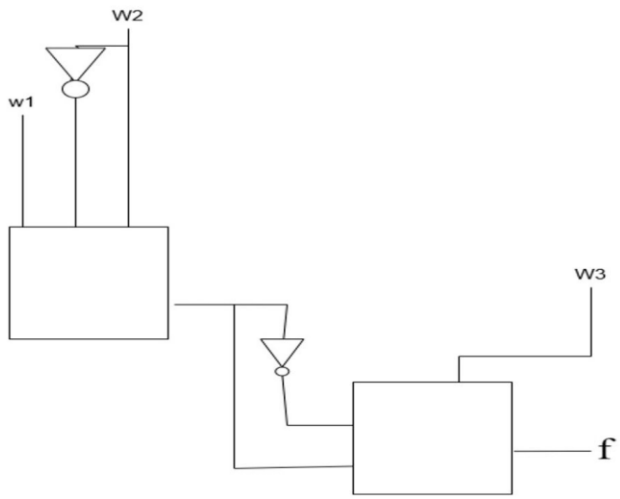
Function f from Table 1a

$$f = \overline{w_1 \oplus w_2 \oplus w_3}$$

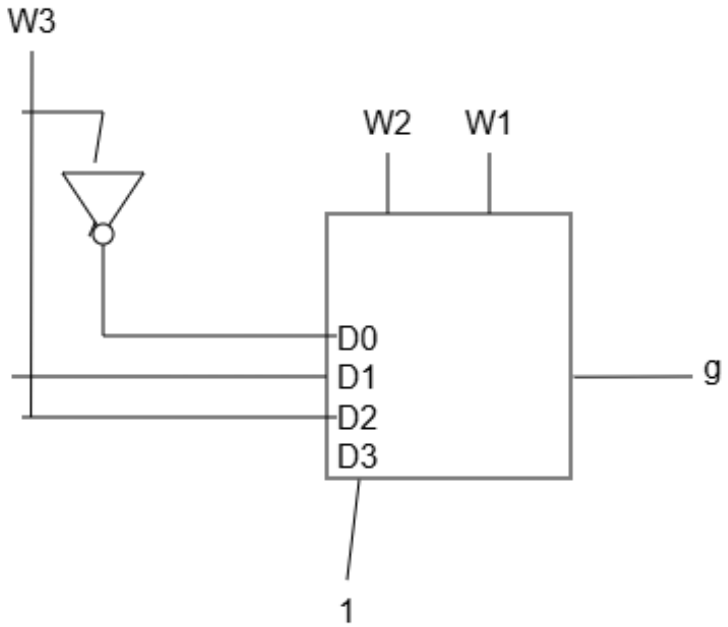
4-to-1 Multiplexer Implementing the Function f: Since the 4:1 MUX contains two select signals, we will use w1 and w2. We can use an inverter to get ~ w3, then we can use 2 w3 and 2 ~ w3 as inputs and see what happens.



2-to-1 Multiplexer Implementing the Function f: Used the same logic as 4-to-1 for 2-to-1



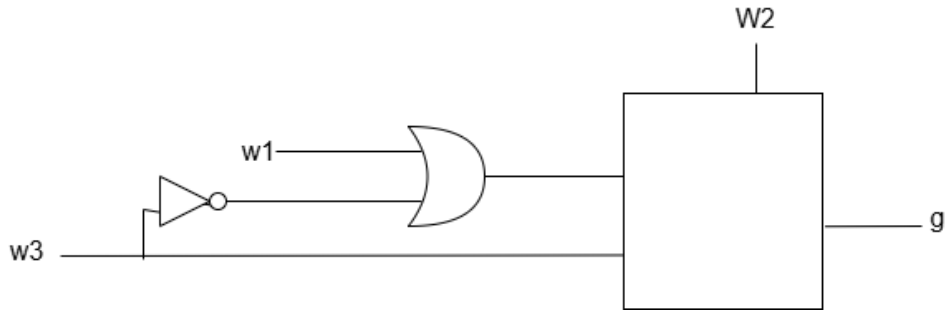
4-to-1 Multiplexer Implementing the Function g: Since the 4:1 MUX contains two select signals, we will use w1 and w2. We can use an inverter to get ~ w3, then we can use 2 w3 and 2 ~w3 as inputs and see what happens.



Truth Table:

s_0	s_1	Input	g
0	0	D_0	w_3
0	1	D_1	w_3
1	0	D_2	$\overline{w_3}$
1	1	D_3	1

2-to-1 Multiplexer Implementing the Function g: In order to construct the identical function, we can choose w2 as the select signal and w3 as one of our inputs. For our second input, we must do w1+w3.



$D_2(w_1 + \overline{w_3})$	$D_1(w_3)$	$s(w_2)$	g
0		0	0
1		0	1
	0	0	0
	1	0	1

3-to-8 Using 2-to-4 SystemVerilog

```
module g_func (
   input  logic w1, w2, w3,
   output logic g
);
   always_comb begin
      if (!w3) g = w1 ^  w2;  // XOR
      else     g = w1 |  w2;  // OR
   end
endmodule
```

3-to-8 Using 2-to-4 TestBench

```
module tb_g_func;
   logic w1,w2,w3,g;
   g_func dut(.w1,.w2,.w3,.g);
   initial begin
      for (int v=0; v<8; v++) begin
         {w1,w2,w3} = v[2:0];
         #1;
      end
      $finish;
   end
endmodule
```

2) Decoder

Truth Table (2-to-4)

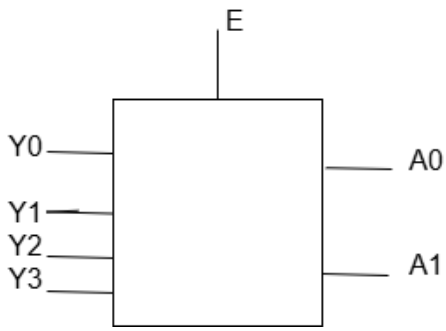
EN	A_1	A_0	Y_0	Y_1	Y_2	Y_3
0			0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

Boolean Equation:

$$Y_0 = EN.\overline{A_1}.\overline{A_0}$$

$$Y_1 = EN.\overline{A_1}.A_0$$

$$Y_2 = EN.A_1.\overline{A_0}$$

$$Y_3 = EN.A_1.A_0$$

Graphic Symbol